TECHNICAL REPORT

Optimal Impression Allocation

How a mobile advertising platform decides, in real time, which campaign each subscriber sees, and why planning those decisions globally, rather than greedily, is where the revenue is.

ABSTRACT

A mobile advertising platform serves impressions under hard constraints: each campaign carries a contracted impression quota that must be delivered within its flight; targeting restricts every campaign to specific audiences, time windows, and content categories; per-subscriber frequency caps bound exposures across all inventory; subscribers arrive in unpredictable order; and each decision must complete within the latency of a page load. Under these constraints, serving each impression to the campaign the viewer is most likely to click maximizes nothing: every impression granted to one campaign is denied to all others. This report presents a two-stage architecture that treats ad serving as a global allocation problem, set in the environment best equipped to solve it: a mobile network operator, whose verified subscriber profiles, complete gateway-observed browsing histories, and single stable identity per subscriber support finer audience segmentation and census-grade traffic forecasts than any publisher-side system. An offline stage clusters subscribers by profile and response behavior, estimates click probabilities per campaign and audience segment, forecasts traffic, and solves a transportation-style linear program that allocates every campaign's quota across segments for maximum expected revenue. The resulting plan is compiled into bit-masked lookup tables that an online stage executes per impression in constant time, blending cluster-level click rates through soft membership weights and steering delivery back to the planned trajectory with a pacing index. On a minimal worked instance, planned allocation raises expected clicks 11% over greedy serving with identical inputs. In Monte-Carlo simulation at realistic scale (25 campaigns, 60 segments, 200,000 impressions per run, 20 replications), the planned policy delivers a mean 6.5% revenue lift and 33% more expected clicks than greedy serving on identical event streams, and captures

94% of the clairvoyant offline optimum despite serving against forecast rather than realized traffic. Served responses feed the next planning cycle, closing the loop.

1 Summary

A mobile advertising platform must answer one question millions of times a day: *a subscriber has just opened a page; which campaign should fill this impression?* The obvious answer, show whichever ad this subscriber is most likely to click, is wrong. Campaigns carry contracted impression quotas, subscribers arrive in an unpredictable order, and every impression given to one campaign is an impression denied to all others. Serving each event myopically fills popular campaigns with the first users who arrive, not the users who respond best, and leaves the remaining campaigns with a poorly matched audience.

This report describes an architecture that treats ad serving as a **global allocation problem**. An offline analytics stage learns who responds to what, clusters subscribers by profile and response behavior, predicts traffic per audience segment, and solves for an allocation plan that maximizes expected revenue across all campaigns at once. The plan is compiled into compact lookup tables. An online decision stage then serves each impression in constant time by consulting those tables, tracking actual delivery against plan, and steering each campaign back toward its planned trajectory. Feedback from served impressions flows back to analytics, closing the loop and refining the model each cycle.

2 The operator’s vantage point

Mobile advertising delivered by the network operator itself is a different discipline from advertising on the open web, and the difference is the data. A web ad network observes a browser: an unverified, resettable cookie, activity on the sites it happens to instrument, and demographics inferred by guesswork. A network operator observes a *customer*. The subscriber relationship supplies a verified identity anchored in a billing account, declared attributes such as age, gender, and home address, and financial signals such as plan type, spending level, and purchase history for content like music and ringtones. The network itself supplies what no publisher-side system can assemble: every browsing session crosses the operator’s gateway regardless of destination site, so the response history is complete rather

than sampled from one publisher's corner of the web, and it is joined under one stable identifier rather than stitched probabilistically across cookies and devices.

Dimension	Web ad network	Network operator
Identity	Cookie or device fingerprint; resettable, unverified	Subscriber account; stable, verified, one per person
Demographics	Inferred from behavior	Declared at subscription: age, gender, address
Financial signal	None or modeled	Plan, spend level, content purchase history
Location	Coarse, via IP	Network-observed, continuous
Browsing coverage	Sites the network instruments	All traffic through the gateway
Exposure control	Per-site frequency caps	Global caps across all inventory, per subscriber

This vantage point changes what is *possible*, and each advantage maps directly onto a component of the architecture in this report. Verified profiles and complete response histories make the proximity metric of Section 5.2 meaningful: subscribers are clustered on real attributes and full behavioral records, not on noisy proxies, so the clusters are stable enough to pool click statistics over. Complete traffic visibility makes the context slots and traffic forecasts of Section 5.3 accurate: the operator sees the whole denominator, every impression opportunity, not just the ones flowing through partnered sites. The single stable identifier makes global exposure policy enforceable: a frequency cap means the subscriber saw at most so many ads today *anywhere*, which no cookie-scoped system can promise. And the billing relationship closes the loop commercially, since the same account that receives the ad can redeem the offer.

Two consequences deserve emphasis, because the allocation machinery depends on them quantitatively. First, **segmentation can be far finer**. Cluster granularity is limited by statistics: a segment must generate enough impressions and clicks for its estimated rates to be trustworthy. Because the operator's data is complete (every subscriber, every session) and clean (verified attributes, deduplicated identity), the same statistical reliability is reached with

much smaller segments than inferred web data allows. Finer segments mean click probabilities that track real behavioral differences instead of averaging over them, and every refinement of $p_{C,S}$ translates directly into allocation-plan revenue. Second, **overall load is predictable**. The operator observes total traffic, its daily and weekly rhythm, per content category and per region, so the supply forecast H_S is estimated from a census rather than a sample. Accurate supply forecasts are what make an offline plan trustworthy at all: a plan allocating impressions that never materialize, or missing volume that does, must be repaired constantly at serving time. Rich subscriber knowledge sharpens both sides of the optimization, the value of each pairing and the volume available to pair.

The vantage point also carries obligations that shape the design. Not every subscriber has a profile: prepaid users may be anonymous, which is why the clustering machinery supports membership computed from response behavior alone (Section 5.3). And precisely because the data is rich, personal attributes never leave the offline stage in raw form; what reaches the serving path is a set of anonymous cluster distances and campaign eligibility lists, not the profile itself.

Rich knowledge, however, only determines how well each individual impression *could* perform. It says nothing about how impressions should be divided among competing campaigns. That is a separate problem, and it is where the revenue argument begins.

3 Motivation: the cost of deciding one impression at a time

Consider the smallest interesting instance. Two campaigns run at the same time: a soft drink and a sports car. Each has bought exactly **two impressions**. Four subscribers will each generate one ad opportunity. From past behavior we can estimate every subscriber's probability of clicking each campaign:

Subscriber	P(click | drink)	P(click | car)
u_1	0.10	0.80
u_2	0.30	0.50
u_3	0.20	0.60

Subscriber	P(click | drink)	P(click | car)
u_4	0.40	0.70

A greedy server shows every subscriber their personally best ad. All four prefer the car, so the car campaign's quota is consumed by whoever happens to arrive first, say u_1 and u_2. The drink campaign is then forced onto the remaining two:

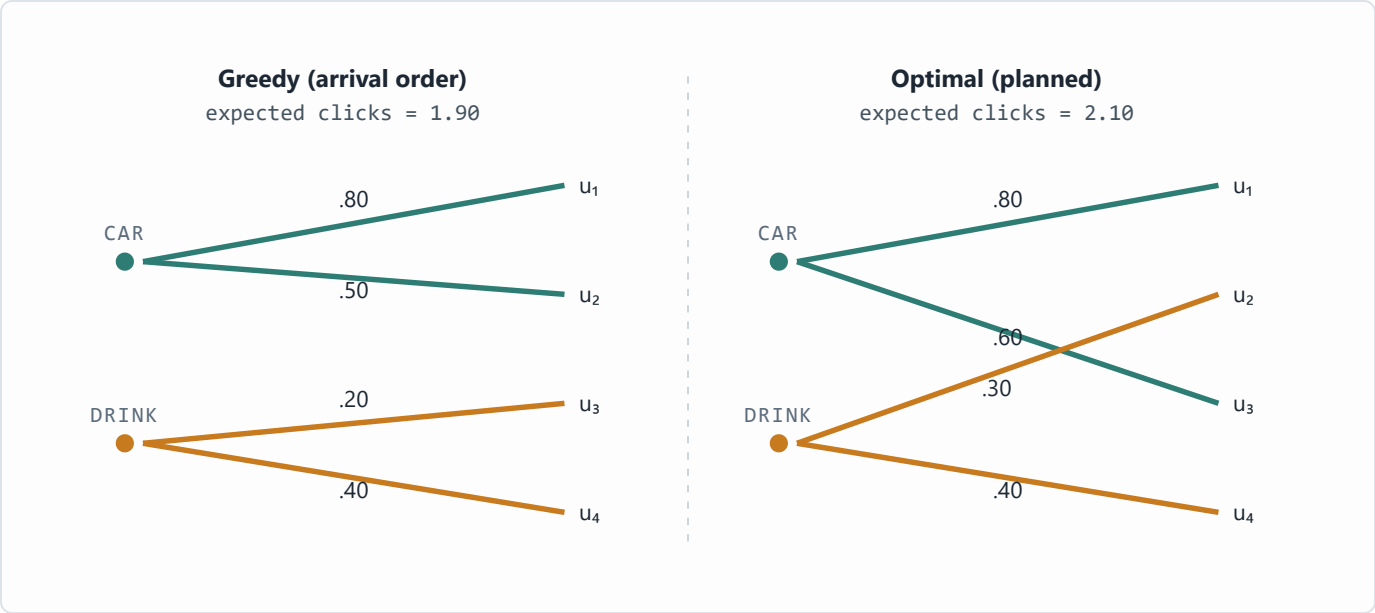

Figure 1. Two campaigns, four subscribers, two impressions per campaign. Greedy serving fills the car campaign with the first arrivals (u_1, u_2) for 0.80 + 0.50 + 0.20 + 0.40 = 1.90 expected clicks. The optimal assignment gives the car campaign u_1 and u_3 and the drink campaign u_2 and u_4, for 0.80 + 0.60 + 0.30 + 0.40 = 2.10, an 11% lift from re-pairing the same impressions.

Nothing about the subscribers or the campaigns changed between the two panels. The only difference is that the right-hand assignment was chosen *jointly*: the car campaign gives up u_2 (a 0.50 click chance) to free that subscriber for the drink campaign (where they are the drink's best audience at 0.30), and takes u_3 (0.60) instead. Each individual swap looks locally worse for someone; the total is strictly better.

The gap grows with scale. With hundreds of campaigns, millions of subscribers, targeting constraints, frequency caps, and time-of-day windows, greedy serving does not just lose a few percent, it systematically starves narrow-audience campaigns, over-exposes indifferent users, and leaves contracted quotas unfilled at the end of the flight. Optimal allocation is not a refinement of ad serving; it is the difference between selling inventory and wasting it.

> **The core insight:** click probability is a property of a *pair* (audience, campaign), not of a campaign alone. Revenue is maximized by choosing the pairing globally, subject to each campaign's quota, and this choice can be made ahead of time because audience behavior is statistically predictable even though individual events are not.

4 Problem formulation

Formally, the planning stage solves a transportation-style optimization. Audience supply is described by segments: s ranges over (cluster, context slot) pairs, where a cluster is a group of behaviorally similar subscribers and a context slot is a recurring situation (a time window crossed with a content category). Let H_s be the predicted number of impressions segment s will generate during the planning horizon, and $p_{c,s}$ the estimated probability that an impression from segment s produces a click on campaign c. Each campaign pays r_c per impression and q_c per click, and has bought Q_c impressions. The plan chooses $x_{c,s}$, the number of impressions of campaign c allocated to segment s:

$$
\begin{aligned}
\text{maximize} \quad & \Sigma_{c,s} \; x_{c,s} \cdot (r_c + q_c \cdot p_{c,s}) \\
\\
\text{subject to} \quad & \Sigma_c \; x_{c,s} \;\le\; H_s \qquad \text{(segment supply)} \\
& \Sigma_s \; x_{c,s} \;=\; Q_c \qquad \text{(campaign quota)} \\
& x_{c,s} = 0 \text{ where targeting excludes } (c, s) \\
& x_{c,s} \;\ge\; 0
\end{aligned}
$$

Figure 1 is this program with two campaigns, four singleton segments, $H_s = 1$ and $Q_c = 2$. At production scale the same structure holds; the segments are just coarser and the counts larger. Because both objective and constraints are linear, the plan is solvable with standard methods even for large instances, and, critically, it is solved **offline**. The online server never optimizes; it executes a pre-computed plan and corrects for drift.

Two practical notes. First, the objective prices impressions and clicks separately, so campaigns that pay mostly per impression and campaigns that pay mostly per click compete in one currency, expected revenue. Second, the quota constraint is what makes greedy

serving inadequate: without it, per-event argmax would be optimal, and no planning stage would be needed.

5 The offline stage: learning the inputs to the plan

The allocation program needs two inputs that do not exist in raw data: predicted traffic per segment (H_s) and click probabilities per campaign-segment pair ($p_{C,S}$). The offline analytics stage manufactures both from event history, under three modeling assumptions, each holding with high probability rather than certainty:

1. The distribution of (context, profile) pairs observed in the past will recur in the future.
2. Users who responded similarly under similar conditions will continue to respond similarly.
3. Users who respond similarly share something observable in their profiles or histories.

Assumption 1 justifies forecasting traffic from history; assumptions 2 and 3 justify pooling sparse per-user click data into clusters that have enough volume to estimate probabilities reliably.

5.1 Response records

The atomic unit of evidence is an ad response record: who saw what, in what context, and whether they clicked.

Time of day	Day of week	Content category	Campaign	Click
`11:23`	Monday	News	`C1 (active)`	`0`
`14:15`	Monday	Sport	`C2 (active)`	`1`
`18:14`	Sunday	Sport	`C4 (past)`	`1`

Records are grouped per subscriber and joined with the profile (age, gender, location, income band, purchase indicators) where one is available. Past campaigns matter as much as active ones: a subscriber's reaction to a finished campaign is evidence about how they will react to similar future ones.

5.2 Proximity and clustering

Subscribers are clustered by a weighted distance that mixes profile similarity with response-pattern similarity, response to the same campaigns (active and past) and response to similar campaigns. Each attribute contributes its own distance component:

Attribute	Distance component
Age	Absolute difference
Gender	Fixed penalty if different
Campaign affinity	By campaign category; fixed penalty if different
Day of week	Absolute difference, circular
Location (x, y)	Straight-line distance

Membership is deliberately **soft**. A subscriber is not filed into one cluster; they carry a pre-computed distance to each nearby cluster. A 35-year-old married music buyer in a large city may sit at distance 0.8 from the "metropolitan professionals" cluster and 0.1 from "music lovers"; a teenage ringtone buyer in the same city sits close to "kids" and "music lovers" instead. These distances become interpolation weights at serving time (Section 6), so predictions blend the behavior of every cluster the subscriber resembles.

5.3 Cluster representation and click estimation

Each cluster is summarized by two compact structures. Its *context slots* describe when and where the cluster generates traffic, with predicted hit counts, this is the supply side H_s. Its *click statistics* record impressions and clicks per campaign per context slot, the raw material for $p_{c,s}$:

Context slot	Time of day	Day of week	Category	Predicted hits
CS1	10:00–12:00	Mon–Tue	News, Sport	3,000
CS2	11:00–17:00	Sun	Sport	4,000

Campaign	Context slot	Impressions	Clicks
C1	CS1	1,000	3

Campaign	Context slot	Impressions	Clicks
C1	CS2	2,000	2
C2	CS1	1,000	7

For a brand-new campaign with no history of its own, probabilities are borrowed from the most similar past or active campaigns in the same category, the same pooling idea applied across campaigns instead of across users. Where the profile itself is missing (anonymous or prepaid subscribers), distance to clusters is computed from response history alone, and the subscriber inherits the response statistics of the cluster they behave like. The design degrades gracefully as data thins out: per-user evidence backs off to cluster evidence, and per-campaign evidence backs off to category evidence.

5.4 Compiling the plan into lookup tables

Solving the allocation program yields planned impression counts per (campaign, cluster, context slot). Everything the online server needs is then flattened into three lookup structures, so that no clustering, no distance computation, and no optimization happens on the serving path:

Structure	Keyed by	Contents
Model table	`cluster × campaign`	Targeted time slots and content categories as bit masks; planned impressions; expected clicks
Match table	`subscriber`	Campaigns this subscriber is eligible for under targeting rules
Cluster table	`subscriber`	Distance to each nearby cluster (the soft-membership weights)

Time and content targeting are encoded as fixed-width bit masks, 168 weekly hour-slots and up to 256 content categories, so an eligibility test at serving time is a bitwise AND rather than a rule evaluation. A model table row reads, in full: *"cluster L1, campaign C1, time mask 0100…, content mask 1111…, 10,000 planned impressions, 450 expected clicks."* That one row carries targeting, allocation, and the performance baseline the online stage will pace against.

6 The online stage: executing the plan in real time

At serving time an impression event arrives carrying a subscriber identifier and a context (time, day, content category of the page being viewed). The decision engine must return a single campaign within the latency budget of a page load. It consults the three model structures plus two live state tables: a **status table** mirroring the model table but accumulating *actual* impressions and clicks, and a **response table** holding each subscriber's recent exposure records.

Figure 2. The closed loop. Offline analytics compiles the model and allocation plan into lookup tables; the online engine executes them per impression and streams response records back, which the next analytics cycle uses to refine clusters, probabilities, and the plan.

6.1 The seven-step decision

1. **Find eligible campaigns**

   `match table` Look up the campaigns this subscriber may be shown under targeting rules.

2. **Apply exposure policy**

   `response table` Drop campaigns that would violate frequency rules for this subscriber, a minimum interval between exposures, a maximum number of exposures per day.

3. **Fetch cluster distances**

`cluster table` Retrieve the subscriber's pre-computed distances to nearby clusters.

4 **Predict click probability per candidate**

`model table` Blend cluster-level click rates with the soft-membership weights (Section 6.2).

5 **Compute pacing per candidate**

`status table` Compare actual delivery against plan, prorated to this point in the flight (Section 6.3).

6 **Decide**

If any campaign is critically underserved, serve it. Otherwise drop overserved campaigns and serve the candidate with the greatest expected revenue.

7 **Update state**

`status + response tables` Increment the served campaign's actual counters and append the exposure record, which later feeds back to analytics.

Every step is a keyed lookup, a bit-mask test, or arithmetic over a handful of rows. The expensive work, clustering, probability estimation, optimization, was all paid for offline.

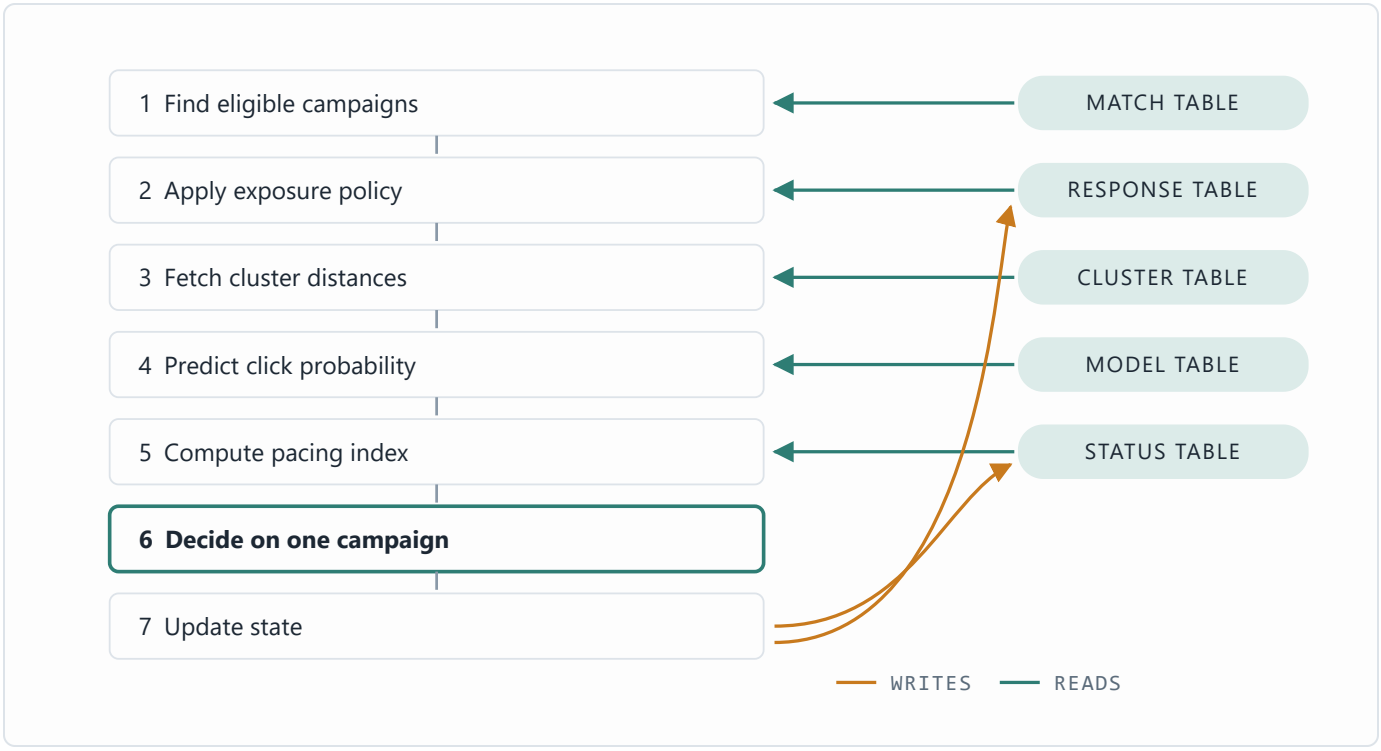

Figure 3. The seven-step decision path and its data dependencies. Steps 1–5 each read one pre-computed or live table; step 6 is pure arithmetic over the candidates; step 7 writes the served impression back into the status and response tables, which is also the record that feeds the next offline refinement cycle.

6.2 Click prediction by soft membership

Suppose the event context is *Monday, 14:00, Sport* and the subscriber sits at weight 0.2 to cluster L1 and 0.8 to cluster L2 (weights derived from the stored distances). The model table gives campaign C1's historical rates in the matching context: 40 clicks per 8,000 impressions in L1 and 60 per 10,000 in L2. The prediction is the weighted blend:

```
p(click | C1)  =  0.2 · (40 / 8,000)  +  0.8 · (60 / 10,000)  =  0.0058
```

Because the subscriber borrows from every cluster they resemble, the estimate is stable even for users with thin personal histories, and it is computed from two table rows and four multiplications.

6.3 Pacing: planned versus actual

The plan is a trajectory, not just a total. If a campaign is one third of the way through its flight in a given cluster, it should have received roughly one third of its planned impressions there. The engine aggregates this across the clusters where the campaign runs:

Cluster	Flight elapsed	Planned by now	Actual
L1	33%	8,000 × 0.33 = 2,640	2,000
L2	50%	10,000 × 0.50 = 5,000	5,100
Total		**7,640**	**7,100**

```
performance index  =  actual / planned-by-now  =  7,100 / 7,640  ≈  93%
```

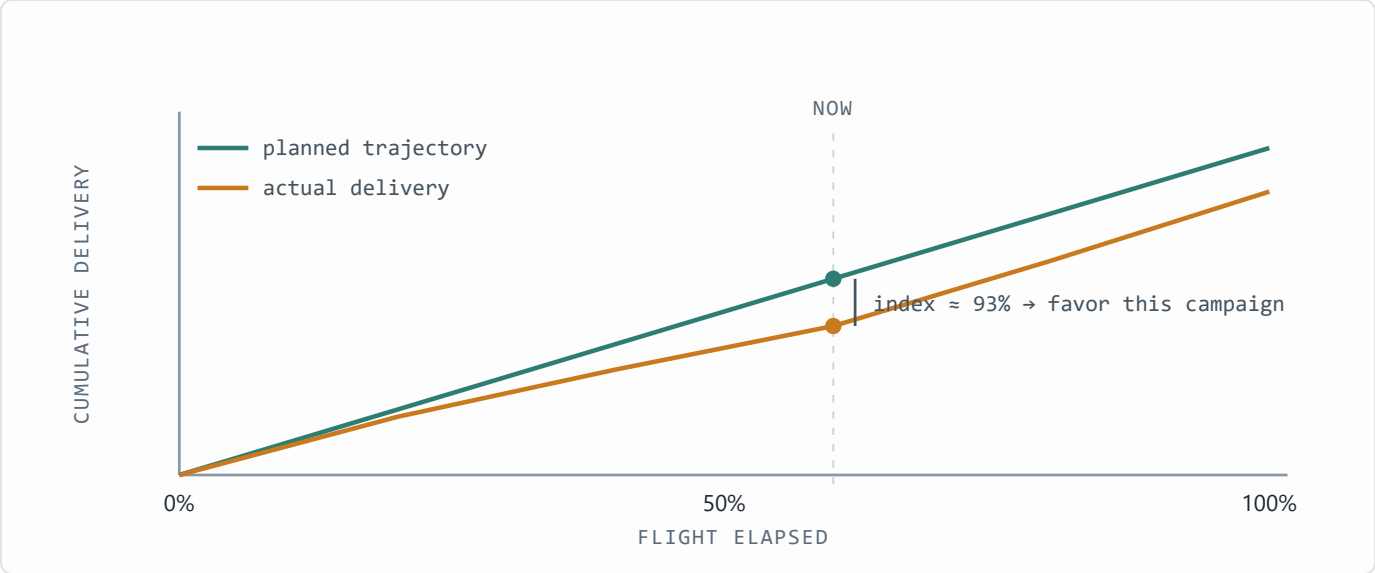

Figure 4. Pacing as trajectory tracking. The plan implies a cumulative delivery path; actual delivery drifts below it, the performance index measures the gap at the current moment, and the decision rule favors the lagging campaign on suitable impressions until the curves reconverge.

An index near 100% means the campaign is on plan. Far below a threshold, the campaign is critically underserved and jumps the queue (step 6); far above, it is overserved and stands aside. Between the thresholds, campaigns compete on expected revenue:

Campaign	Click price	Impression price	Perf. index	P(click)	Expected revenue
C1	$0.05	$0.10	55%	0.01	$0.0501
C2	$0.04	$0.30	91%	0.03	$0.0403
C3	$0.06	$0.20	96%	0.02	$0.0604
C4	$0.05	$0.10	105%	0.07	$0.0507

Here no campaign is critically underserved and none has crossed the overserved threshold, so the engine serves C3, the greatest expected revenue for this impression. The pacing mechanism is what lets a static offline plan survive contact with stochastic reality: the plan sets the destination, and the index steers thousands of small per-impression corrections toward it. Deviations that pacing cannot absorb, a traffic forecast that proves wrong, a campaign that outperforms its category prior, are exactly what the feedback loop carries back into the next planning cycle.

7 Simulation: planned allocation versus greedy serving

The four-subscriber example of Section 3 proves the mechanism exists; a Monte-Carlo simulation quantifies it at realistic scale. Each replication generates a synthetic market: 60 audience segments with lognormal traffic (about 200,000 impression opportunities), 25 campaigns whose quotas sell 80% of expected inventory, per-impression and per-click prices drawn independently, and click probabilities from a low-rank affinity model (each campaign genuinely fits some audiences better than others, with rates spanning 0.1%–8%). Each campaign is eligible, by targeting, on roughly 60% of segments.

Two policies then serve the *identical* event stream under identical quotas, prices, probabilities, and targeting. The **greedy** policy is the conventional per-event rule: serve the eligible, quota-remaining campaign with the highest immediate expected revenue. The **planned** policy solves the linear program of Section 4 offline on the traffic forecast, then serves each event by drawing down the planned allocation, falling back to greedy where the plan is locally exhausted. The plan never sees the realized traffic; events are drawn stochastically around the forecast, so the planned policy faces genuine forecast error. A third quantity, the **clairvoyant bound**, is the offline optimum computed on the traffic that actually occurred; no online policy can exceed it.

Measure (20 replications)	Mean	Range
Revenue lift, planned over greedy	+6.5%	+4.1% to +8.8%
Expected-click lift, planned over greedy	+33.0%	+22.9% to +40.6%
Planned revenue as share of clairvoyant bound	93.8%	≥ 91.7%

Measure (20 replications)	Mean	Range
Greedy revenue as share of clairvoyant bound	88.1%	≥ 85.4%
Quota fill, both policies	100%	–

Figure 5. Revenue captured by each policy as a share of the clairvoyant offline optimum, per replication. Planned allocation dominates greedy serving in all 20 replications; the two distributions do not overlap (planned minimum 91.7%, greedy maximum 91.6%).

Both policies fill every quota, so the entire gap is *pairing quality*: the same impressions, sold to the same campaigns, assigned to different audiences. This also explains why the click lift (33%) exceeds the revenue lift (6.5%): per-impression fees are earned identically by both policies once quotas fill, so planning's advantage is concentrated wholly in the click-priced component of revenue. In markets weighted further toward performance pricing, the revenue gap widens toward the click gap.

The simulation carries its own controls. Both policies are verified against the clairvoyant bound in every replication. On degenerate instances where all campaigns share identical click probabilities, so pairing cannot matter, the measured lift is 0.000%, confirming that the gap measures allocation quality and not an artifact of the harness. And the planned policy's 93.8% of a bound computed with perfect hindsight shows that forecast noise costs the plan only a few points, the residual that Section 6's pacing mechanism and the refinement loop exist to absorb.

8 Why this decomposition works

The optimization lives where time is cheap. Global allocation over all campaigns and segments is a large linear program, and it is solved where minutes are available, not milliseconds. The serving path touches five small tables and does arithmetic. This is the classic plan/execute split, applied so that the online system inherits the quality of the offline optimum at lookup cost.

Pooling defeats sparsity. No individual generates enough clicks to estimate their own response rates. Clusters do. Soft membership then re-personalizes the pooled estimates, blending each subscriber's several behavioral neighborhoods instead of forcing a single label. The same back-off logic handles new campaigns (borrow from similar campaigns) and profile-less subscribers (cluster by response history alone), so one mechanism covers three cold-start problems.

The plan is a baseline, not a cage. Real traffic never matches the forecast. By carrying planned counts into the serving state and steering with a performance index, the system tolerates forecast error gracefully: small deviations are corrected impression by impression, and persistent ones are evidence, fed back into the next modeling cycle rather than silently accumulated.

Every representation is serving-shaped. Bit masks make targeting tests branchless; pre-computed distances make personalization a weighted sum; per-(cluster, campaign) rows make pacing a two-row query. The offline stage does not merely produce a model, it compiles one, in the same sense a compiler turns a program into a form the machine executes directly.

9 Scope and assumptions

The architecture optimizes expected revenue for a known campaign portfolio over a planning horizon; it does not address auction-based pricing, real-time bidding against external demand, or creative selection within a campaign. Its guarantees rest on the stationarity assumptions of Section 5: allocation quality degrades in proportion to how far future traffic and response distributions drift from the modeled ones between refinement cycles, which sets the practical requirement for how frequently the offline stage must re-run.

Exposure policy enforcement is per-subscriber and rule-based; richer fatigue models would slot into step 2 without changing the surrounding machinery.

Technical report on a two-stage architecture for revenue-optimal ad impression allocation: offline predictive modeling and planning, online table-driven decisioning with pacing, and a closed feedback loop.